

Advanced Natural Language Processing

Deep Learning: Transfer Learning Encoder-Decoder Attention

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027



Maastricht University

There is more to language than just recurrence

Concepts such as

transfer learning
encoding – decoding
attention

Help us to understand human language
better and faster.

Remember: Word Embeddings for transfer learning

Transfer Learning: understand basics of natural language (morphology, syntax, semantics, etc., and built on top of that to learn new more complex language tasks such as MT, NER, Q&A, etc.)

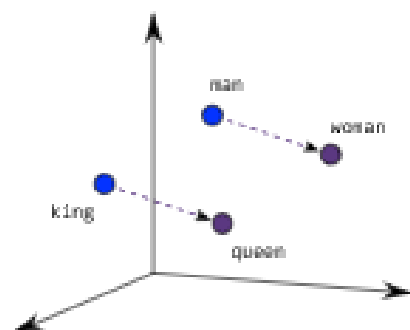


Maastricht University

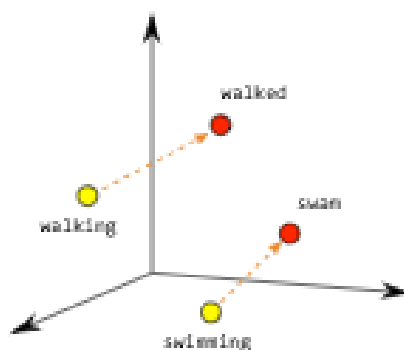
Central Idea to Word Embeddings

- How can we assign an (high-dimensional) encoding to a token that represents morphological, syntactical, semantical and other linguistic properties of a token so that similar linguistic properties result in similar encodings?
- As a result, we should benefit from better linguistic representations and potentially transfer learning properties.

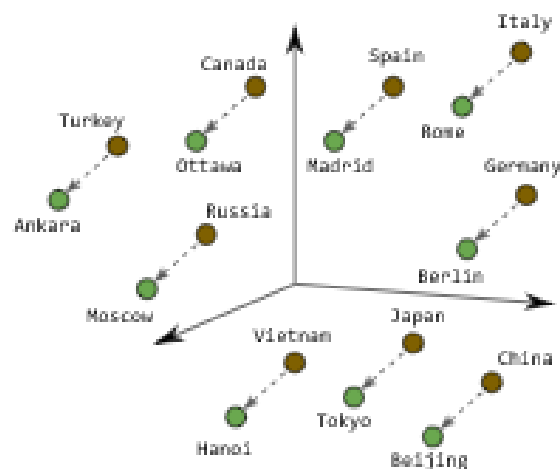
Word Embeddings: similar linguistic context, similar numeric representations



Male-Female



Verb Tense



Country-Capital

Why Word Embeddings?

- Pre-trained language model.
- Understand context better.
- Transfer learning: understand already general aspects of language, subsequent only need to fine-tune for a specific NLP task.
- No need for millions or billions of annotated training data (when using deep learning) when learning a specific linguistic task.

Why Softmax?

Input Vector

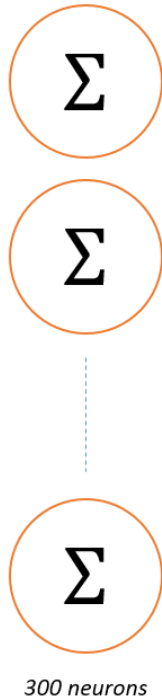


A '1' in the position corresponding to the word "ants"

10,000 positions

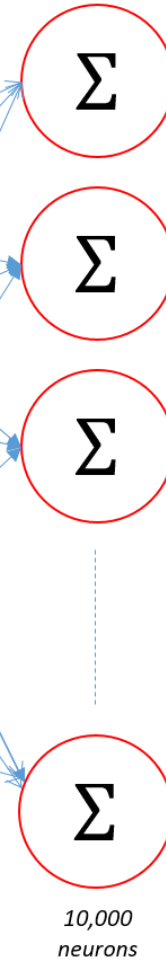
A so-called Hot neuron

Hidden Layer
Linear Neurons



300 neurons

Output Layer
Softmax Classifier



10,000 neurons

Probability that the word at a randomly chosen, nearby position is "abandon"

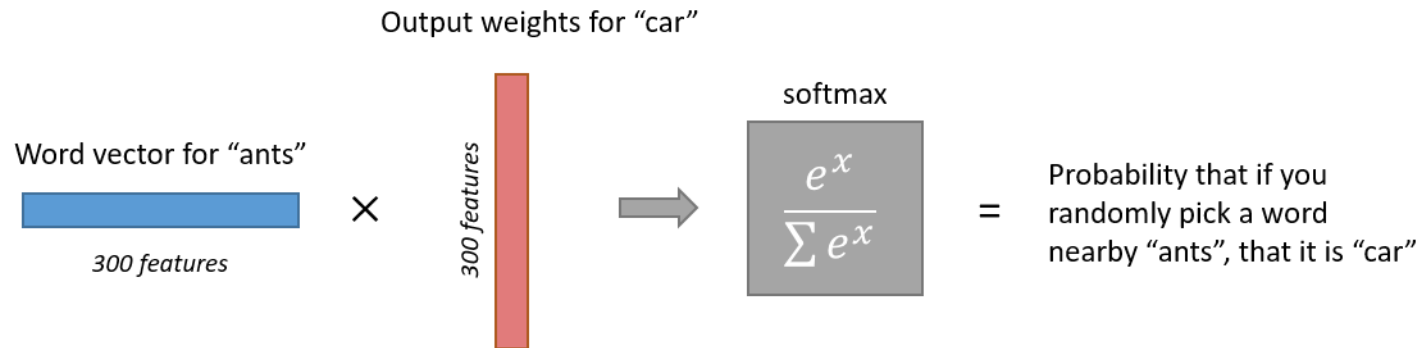
... "ability"

... "able"

... "zone"

The Output Layer

Here's an illustration of calculating the output of the output neuron for the word "car".



The output layer is a *softmax* regression classifier: each output neuron (one per word in our vocabulary!) will produce an output between 0 and 1, and the sum of all these output values will add up to 1.

Each output neuron has a weight vector which it multiplies against the word vector from the hidden layer, then it applies the function $\exp(x)$ to the result. Finally, in order to get the outputs to sum up to 1, this result is divided by the sum of the results from *all* 10,000 output nodes. The results is a probability value.

BERT Embeddings take a similar approach*

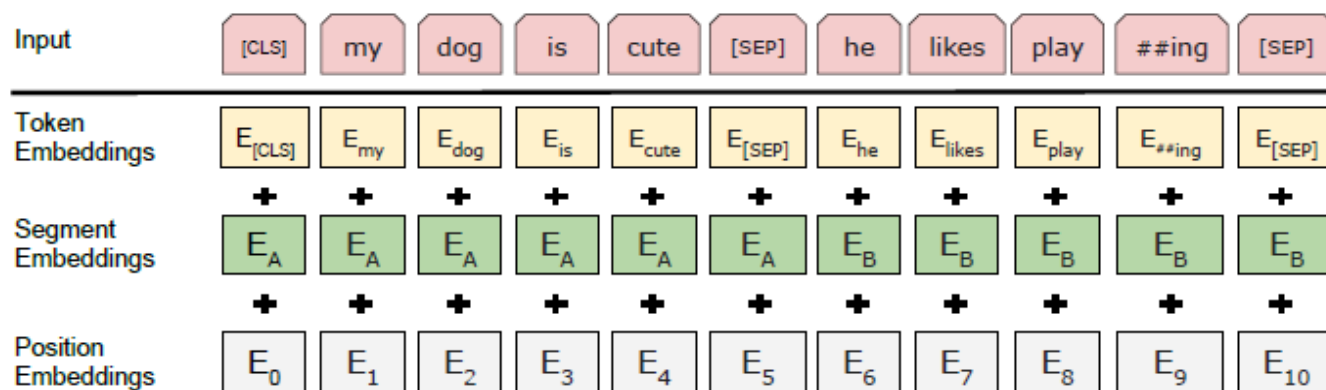


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

* We will take a closer look at BERT in the lecture Deep-Learning for NLP – Part 2 and in the Deep Learning Tutorials.

Context Sensitive Embeddings

Sentences

"bank"
"he eventually sold the shares back to the bank at a premium."
"the bank strongly resisted cutting interest rates."
"the bank will supply and buy back foreign currency."
"the bank is pressing us for repayment of the loan."
"the bank left its lending rates unchanged."
"the river flowed over the bank."
"tall, luxuriant plants grew along the river bank."
"his soldiers were arrayed along the river bank."
"wild flowers adorned the river bank."
"two fox cubs romped playfully on the river bank."
"the jewels were kept in a bank vault."
"you can stow your jewellery away in the bank."
"most of the money was in storage in bank vaults."
"the diamonds are shut away in a bank vault somewhere."
"thieves broke into the bank vault."
"can I bank on your support?"
"you can bank on him to hand you a reasonable bill for your services."
"don't bank on your friends to help you out of trouble."
"you can bank on me when you need money."
"i bank on your help."

The different meanings of the word "bank" will get very different vectors, depending on this context. So, the "financial bank" will be closer to other financial words than the "river bank". This is essential for good language understanding!

mouse (rodent)

mouse (computer)

So, we need different vectors (directions) for different meanings.

The WordPiece Tokenizer

- BERT uses what is called a WordPiece tokenizer.
 - Words are tokenized either into the full forms if it is a known word (part of a 30k vocabulary, then one word becomes one token), or
 - Words are tokenized into word pieces (if it is not part of the 30k vocabulary), then one word can be broken into multiple tokens.

Word	Token(s)
surf	['surf']
surfing	['surf', '##ing']
surfboarding	['surf', '##board', '##ing']
surfboard	['surf', '##board']
snowboard	['snow', '##board']
snowboarding	['snow', '##board', '##ing']
snow	['snow']
snowing	['snow', '##ing']

We will discuss this in more detail in the tutorial



Encoding - Decoding

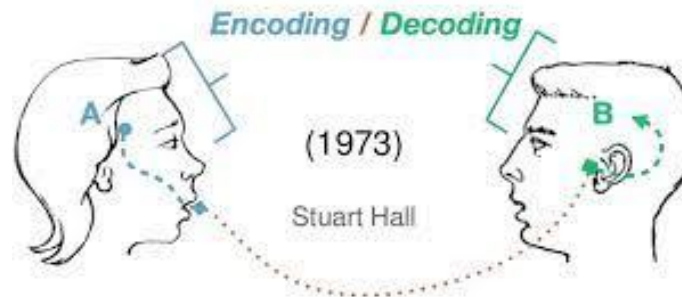


Maastricht University

Teaching note: Think of encoding as “reading and understanding” and decoding as “using that understanding to do something” — translation, Q&A, and summarization are all decoding tasks.

Encoding - Decoding

- Understanding language is an iterative process for humans: When we read or hear a sentence, we process each word accumulating information up to the end of the text. This can be seen as an encoding process.

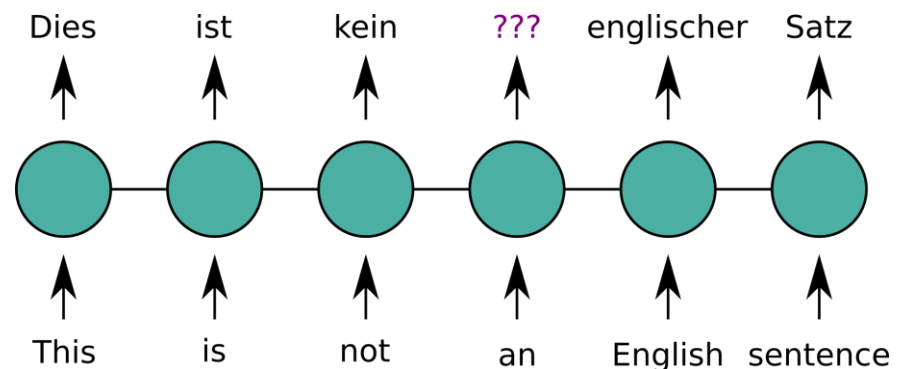
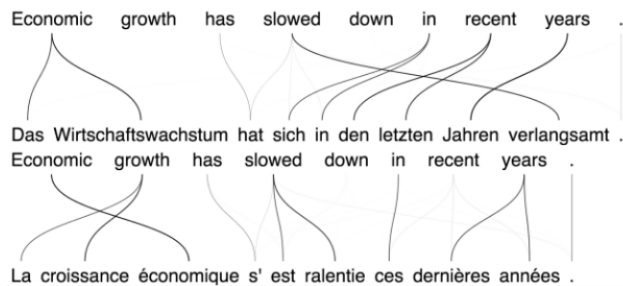


- Then, we perform a certain task: translation, Q&A, abstracting, story telling, ... These can all be seen as decoding processes.

A Naïve Translation model based on a RNN

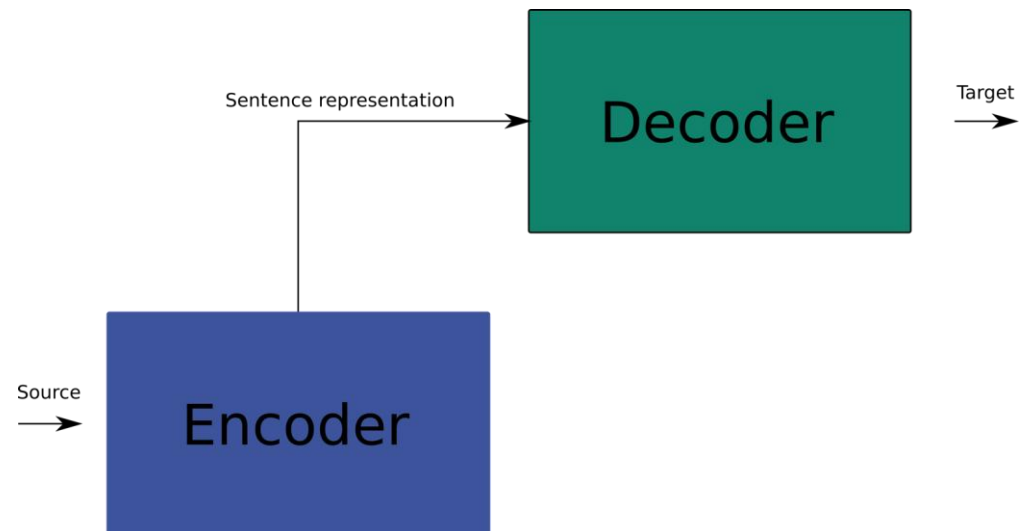
- Predict target word
 - Can store whole history in NN state
 - Translate depends on previous source words
 - Bi-directional: All source Words
- Problems:
 - Target word also depends on other target words
 - Different number of source and target words
 - **Cannot solve the Alignment problem:** sequence change (we will see this in the MT tutorial)

Teaching note: Every problem here is solved by the next two slides: encoder-decoder splits the RNNs apart; attention fixes alignment.



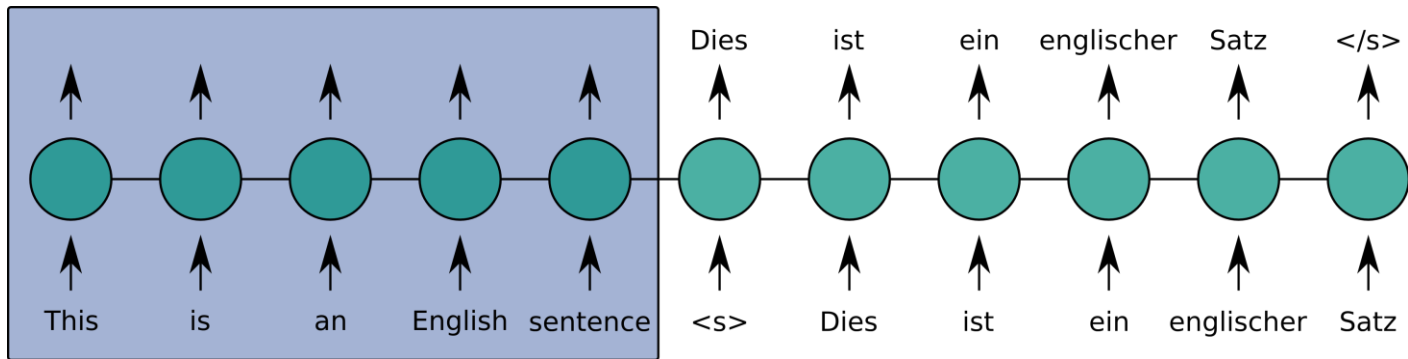
A better solution: Encoder - Decoder

- RNN/LSTM language model
 - Predict words based on previous words
 - Hidden state encodes history
- Sequence to sequence model
 - Predict words based on previous target words and source sentence
 - Encoder
 - Read in source sentence
 - Decoder
 - Generate target sentence word by word



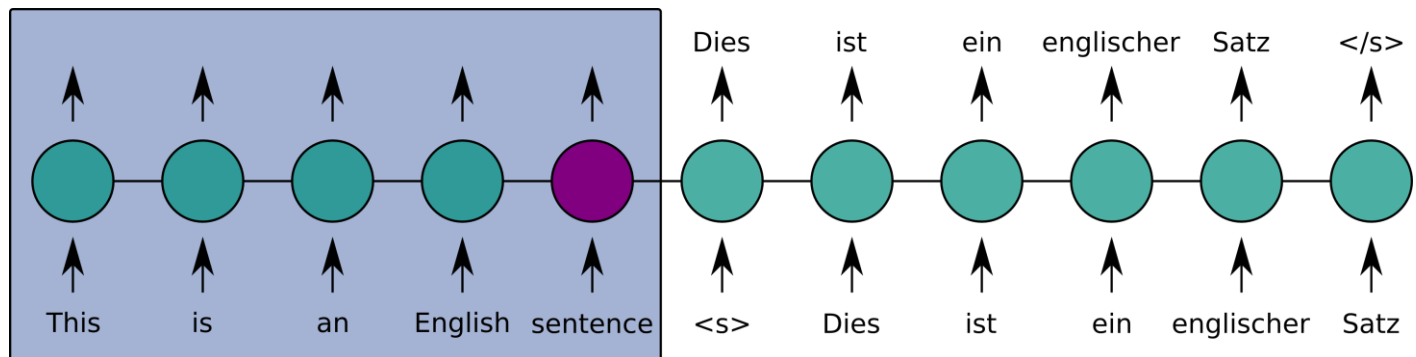
Encoder

- Read in Input
 - Represent content in hidden fixed-dimension vector
- RNN or LSTM-based model
- Ignore outputs of model



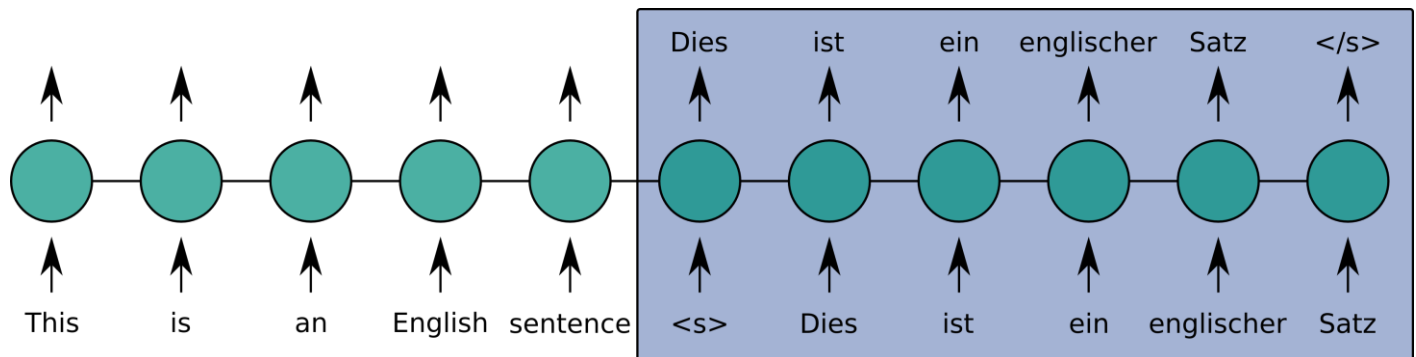
Encoder

- Read in Input
 - Represent content in hidden fix dimension vector
- RNN of LSTM-based model. One RNN/LSTM for encoder, another one for decoder
- Ignore outputs of model
- Memory vector:
 - Store whole content of source sentence



Decoder

- Generate output
 - Use output of encoder as input
- RNN/LSTM-based model. One RNN/LSTM for encoder, another one for decoder
- Input last target word
 - Probability also depends on previous target words
 - Start after begin of sentence symbol



Encoder – Decoder – Translation

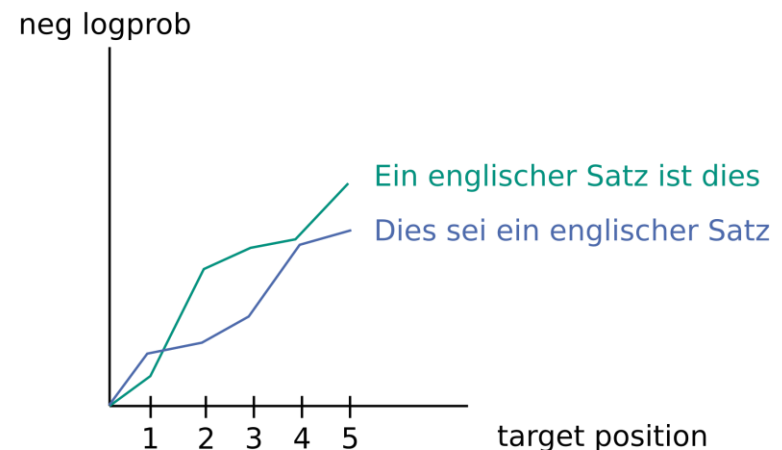
- **Greedy search:**
 - Always select best target word
 - Problem:
 - Might not find most probable sentence



Typical Greedy search from Yoda: “Once you start down the dark path, forever will it dominate your destiny. Consume you, it will.” or “Powerful you have become, the dark side I sense in you.”

- **Sentence probability:**

$$p(e | f) = \prod_{j=1}^n p(e_j | f, e_1^{j-1})$$



Encoder – Decoder – Translation

- **Beam search**

- Always keep best n hypothesis (takes entire sequence into consideration)



How does this relate to the Label Bias Problem in MEMM?

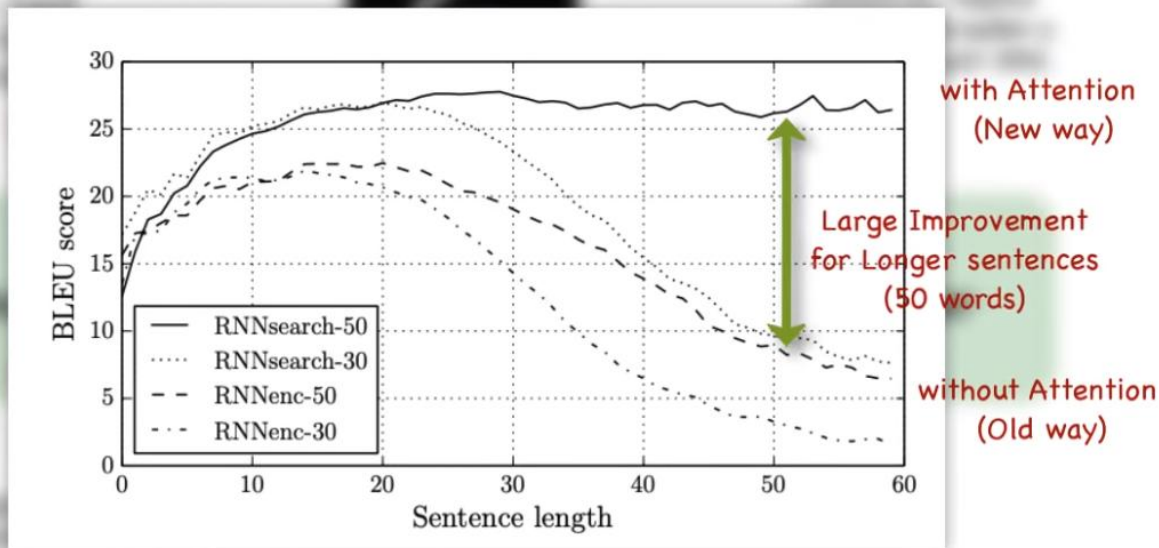
- Neural model estimate the probability very well

- Only small beam needed
- Typical 5-10 hypothesis

We will discuss different decoding strategies in more detail in the lecture on GPT Models

RNN/LSTM Encoder-Decoder Architecture

- Does not work so well with long sentences



Possible reasons:

- The whole source sentence is encoded to one memory vector c in last hidden state of encoder.
- Therefore, we will look into attention mechanisms in the next lecture.

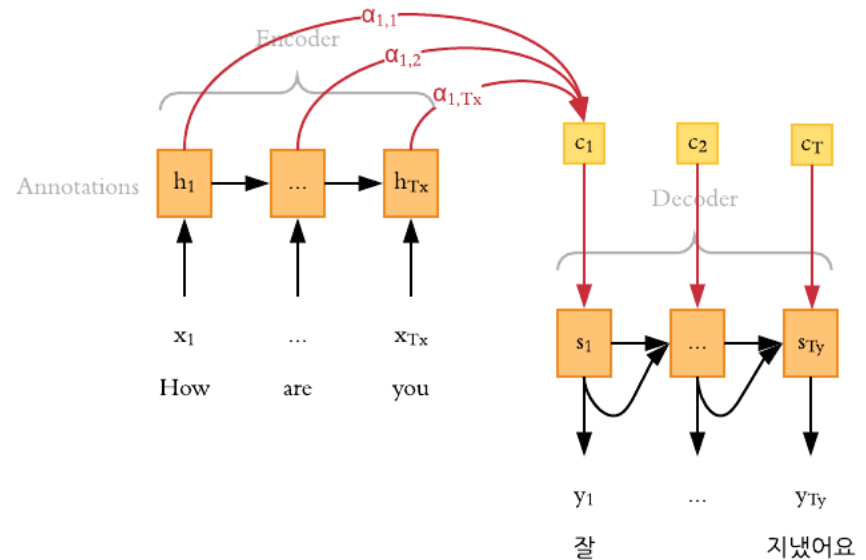
Source: Neural Machine Translation by jointly learning to align & translate (Bahdanau et al., 2016)

Attention in Language



Attention

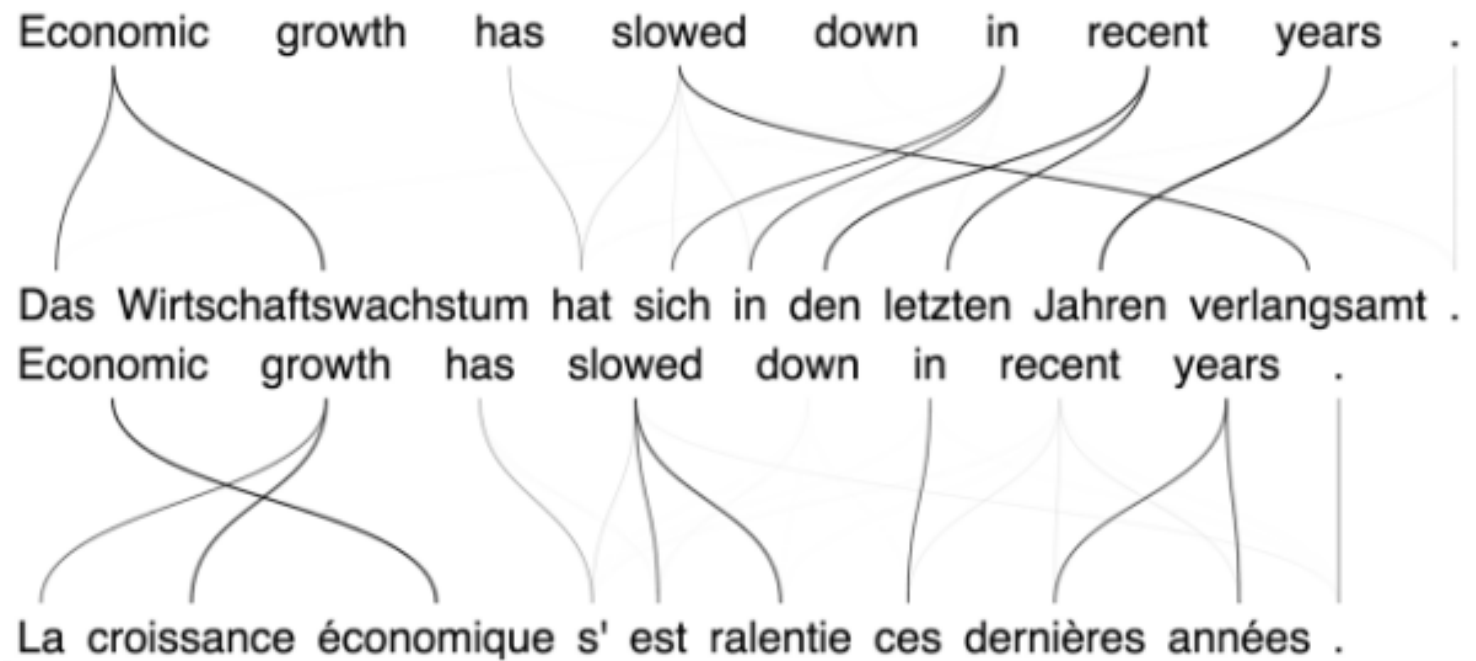
- Attention is the behavioral and cognitive process of selectively concentrating on a discrete aspect of information, whether considered subjective or objective, while ignoring other perceivable information (<https://en.wikipedia.org/wiki/Attention>)
- Attention in natural language processing: “each time the model predicts an output word, it only uses parts of an input where the most relevant information is concentrated instead of an entire sentence. In other words, it only pays attention to some input words”



An illustration of the attention mechanism (RNNSearch) proposed by [Bahdanau, 2014]. Instead of converting the entire input sequence into a single context vector, we create a **separate context vector for each output (target) word**. These vectors consist of the **weighted sums of encoder's hidden states**.

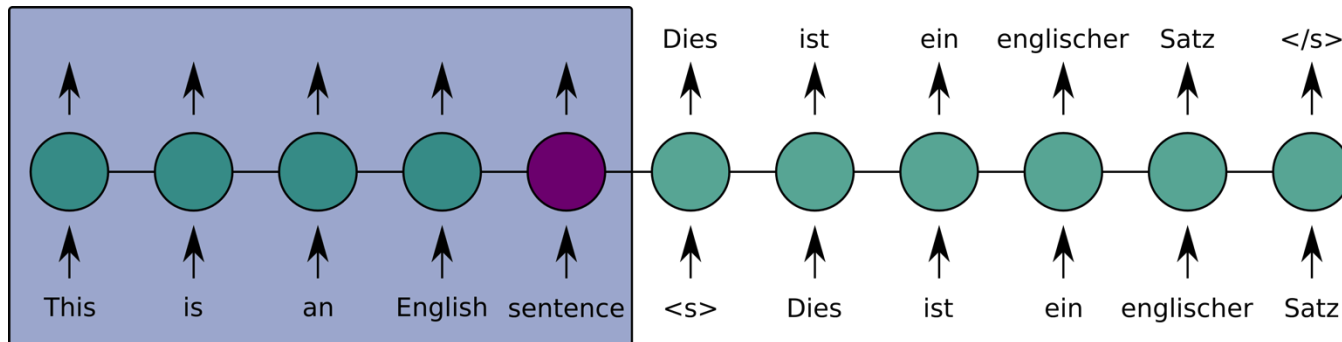
Attention to Deal with Alignment

- Idea:
 - Focus on different parts of the source sentence
 - Different parts are important for different target words
 - Alignment in SMT



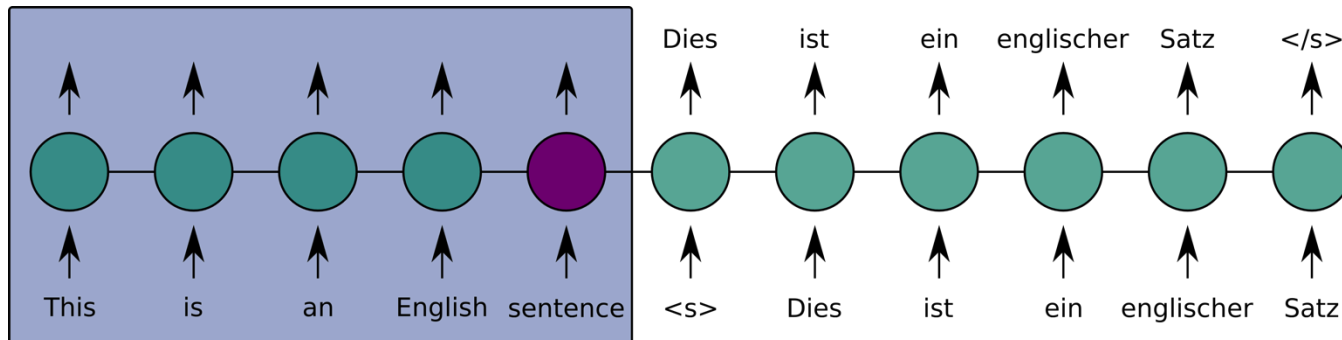
Motivation

- Problem:
 - Bottleneck context vector
 - Fixed size representation of source language



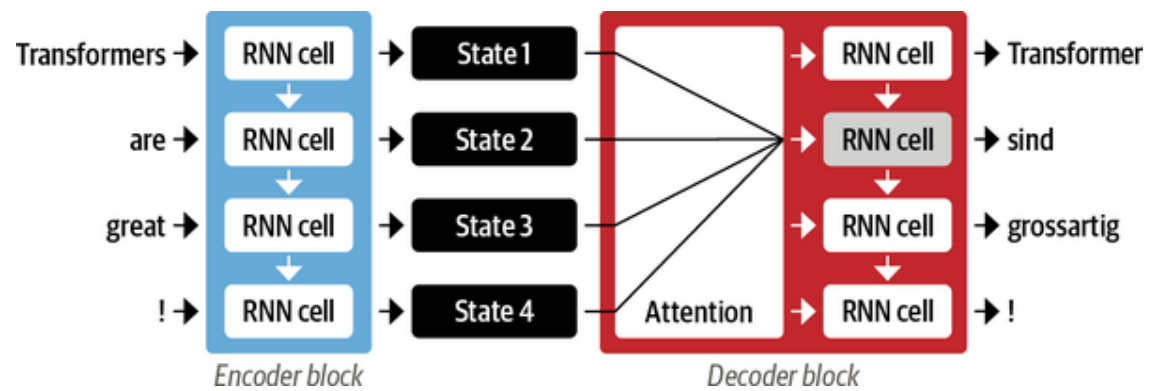
Motivation

- Problem:
 - Bottleneck context vector
 - Fixed size representation of source language
- Idea:
 - Additional input
 - Focus summarization of the source sentence
 - Even more important in text summarization



Attention Mechanisms

- Attention: instead of producing one hidden state for the entire input sequence, the encoder outputs a hidden state at each step.
- In order to prevent too much input: the decoder assigns a different weight (e.g. **attention**) to the encoder states at every decoding timestep.
- By focusing on which input tokens are most relevant at each timestep, these attention-based models can learn nontrivial **alignments and relations** between the words in a generated translation and those in a source sentence.



Teaching note: This is the general recipe used everywhere attention appears later in the course (self-attention, multi-head attention, Transformers): many states in, one weighted summary out.

Attention to look up values from ambiguous data

- Use current element to query data based
- Find similar keys in database
 - Importance:
 - Similarity between query and key
- Select associated values
- Example: How expensive is a burger?

Query: Burger

Key	Value
Cheeseburger	12
Fries	5
Hamburger	10
Ice cream	6

Attention to look up values from ambiguous data

- Use current element to query data based
- Find similar keys in database
 - Importance:
 - Similarity between query and key
- Select associated values
- Example: How expensive is a burger?

“Query: Burger”

Key	Value
Cheeseburger	12
Fries	5
Hamburger	10
Ice cream	6

Attention to look up values form ambiguous data

- Use current element to query data based
- Find similar keys in database
 - Importance:
 - Similarity between query and key
- Select associated values
- Example: How expensive is a burger?

Query: Burger

Key	Value	Similarity
Cheeseburger	12	0.5
Fries	5	0
Hamburger	10	0.5
Ice cream	6	0

Teaching note: This burger example IS attention: query="burger", keys=menu items, values=prices, weights=similarity — exactly Q/K/V in the next slides, just without the matrix notation yet.

Average of sum of the value
times (x) similarity (aka weights)



Attention: where do these value come from?

At every time step t , the decoder LSTM needs to look at encoder states $\{h_1, h_2, \dots, h_n\}$ and decide how much attention to assign to each one.

- Step 1. Create query, keys, and values
 - Query q_t : derived from current decoder hidden state
 - Keys k_i : derived from encoder hidden states
 - Values v_i : usually identical to encoder hidden states

In the formulation used in Transformers later, this is done via learned matrices:

$$q_t = W_Q \cdot h_t^{decoder}$$

$$k_i = W_K \cdot h_i^{encoder}$$

$$v_i = W_V \cdot h_i^{encoder}$$

Where:

- W_Q, W_K, W_V are **learnable weight matrices**
- They are randomly initialized at the start of training, then tuned via backpropagation.

Where do attention values come from?

Step 2. Compute attention scores

Using dot-product or additive attention:

$$score(q_t, k_i)$$

A softmax converts scores to attention weights:

$$\alpha_{t,i} = softmax(score(q_t, k_i))$$

Step 3. Take weighted sum of values

$$context_t = \sum_{i=1}^n \alpha_{t,i} \cdot v_i$$

This **context vector** is fed into the decoder to produce the next word.



Because the model makes predictions (translation / next token), it has a loss L , e.g., cross-entropy.

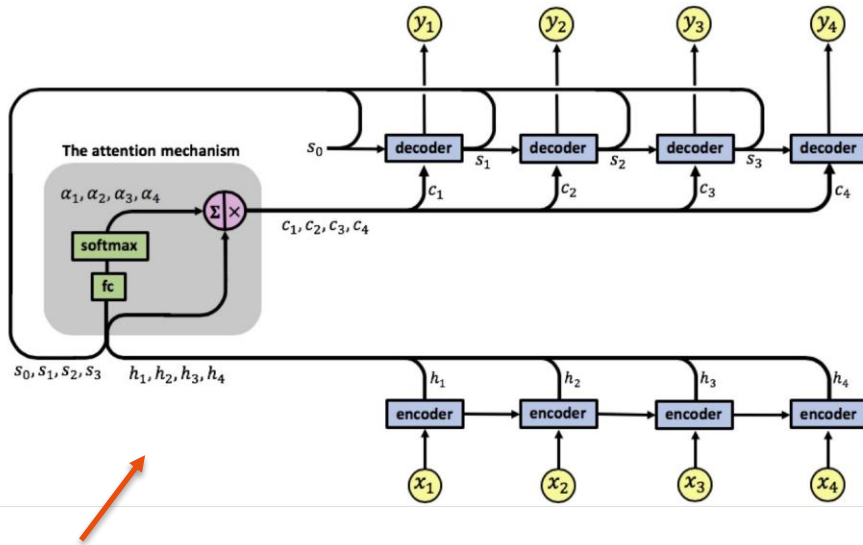
Backpropagation flows from the loss → through the weighted sum → through softmax attention → into W_Q, W_K, W_V .

In effect:

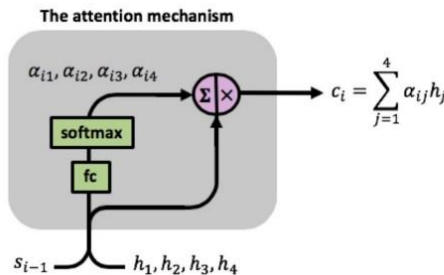
If the model attends to the wrong parts of the sequence, gradients push the matrices so that next time it attends better.

Teaching note: The attention weights are themselves learned by a small trainable network (fc) — nothing about “what to attend to” is hand-coded.

Adding Attention in RNN / LSTM



The attention weights are taught using the attention fully-connected (fc) network and a softmax function



Context vectors:

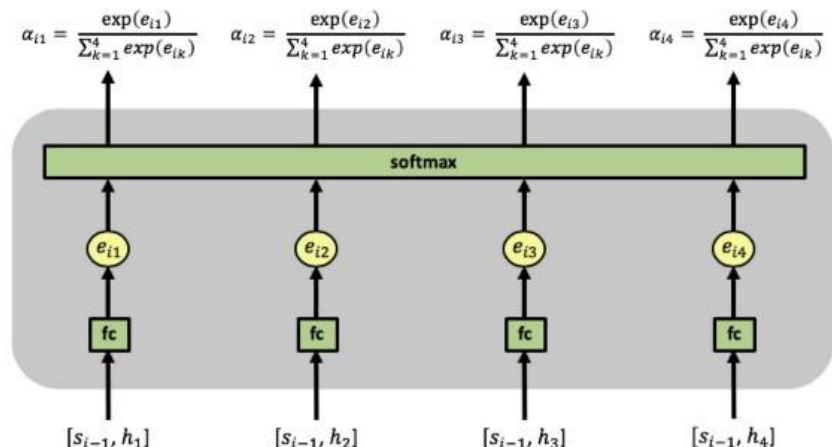
$$c_i = \sum_{j=1}^4 \alpha_{ij} h_j$$

Computation of the attention weights:

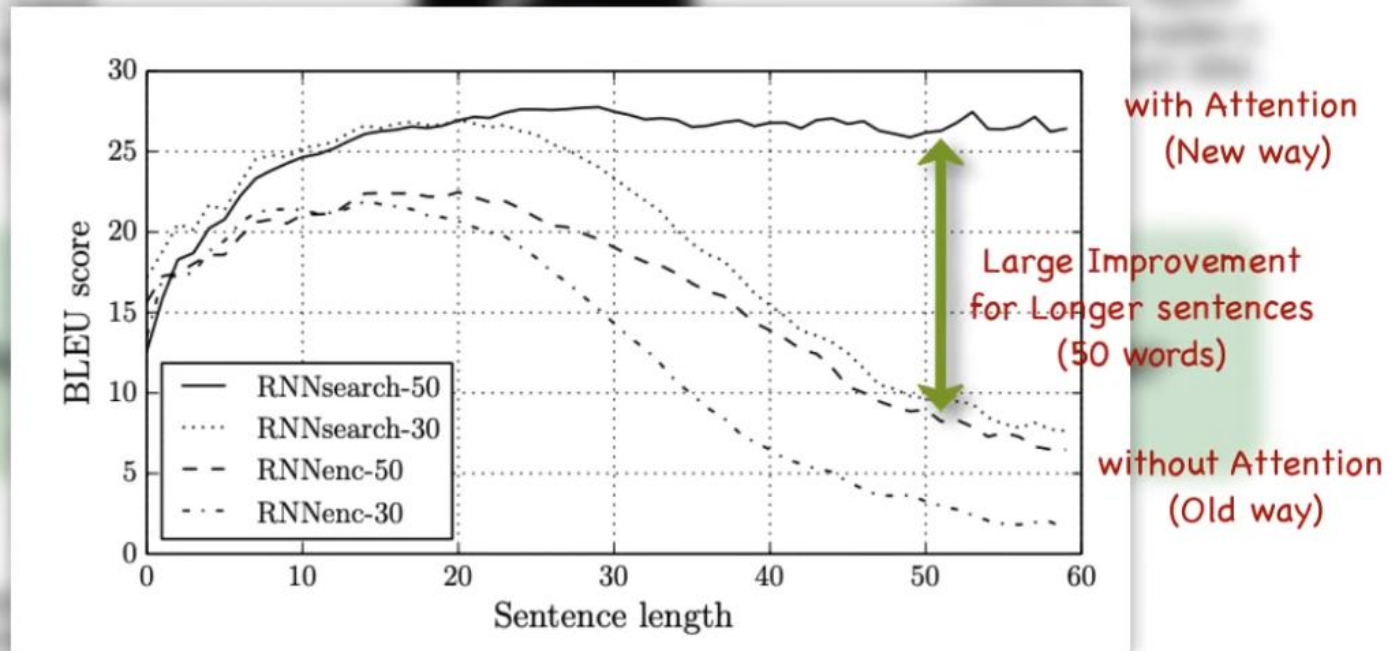
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^4 \exp(e_{ik})}$$

where $e_{ij} = \text{fc}(s_{i-1}, h_j)$

The attention **weights** α_{ij} reflect the importance of h_j with respect to the previous hidden state s_{i-1} in deciding the next state s_i and generating y_i .

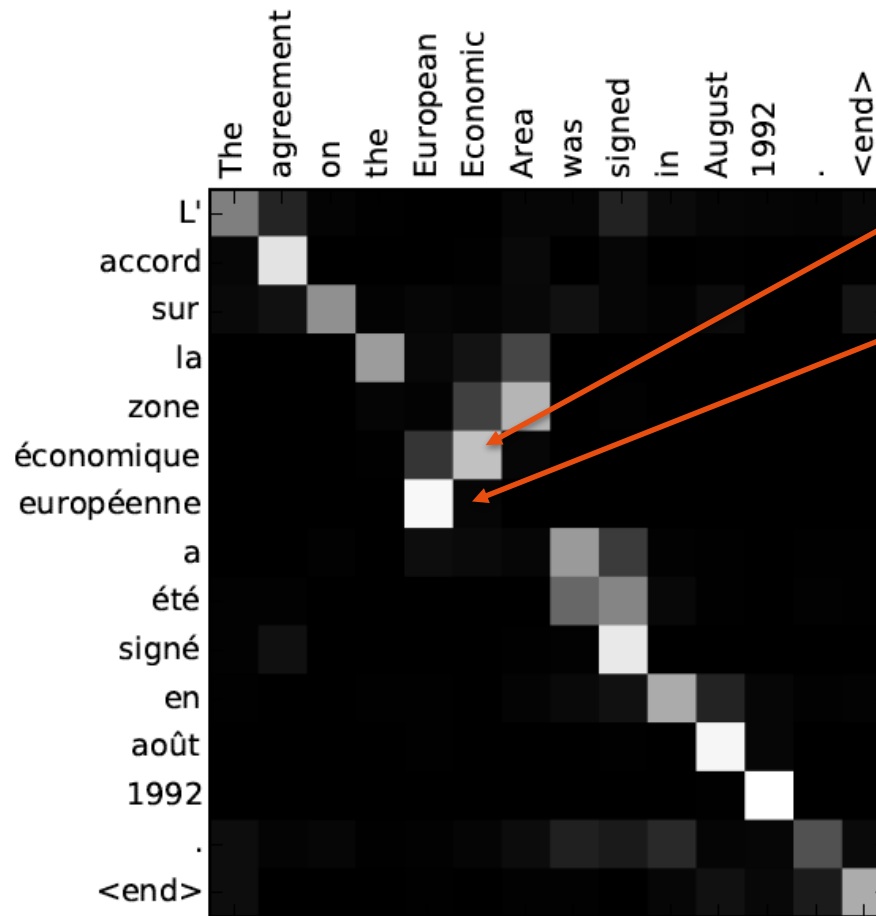


RNN Encoder-Decoder with Attention



Source: Neural Machine Translation by jointly learning to align & translate (Bahdanau et al., 2016)

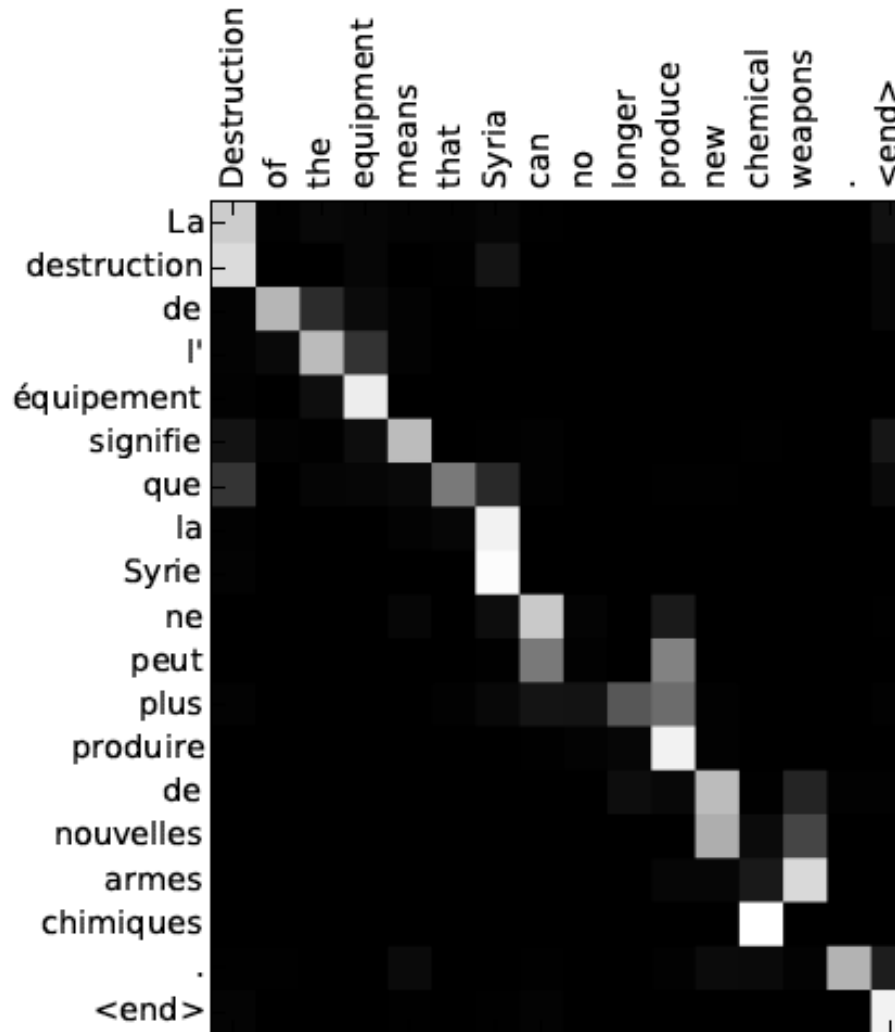
RNN Encoder-Decoder with Attention



Teaching note: Off-diagonal bright cells = the model reordering words on its own — the alignment problem greedy translation couldn't solve (slide 14).

- Light is HIGH attention
- Dark is lower attention
- Black is no attention
- See how attention solves the alignment problem!

RNN Encoder-Decoder with Attention



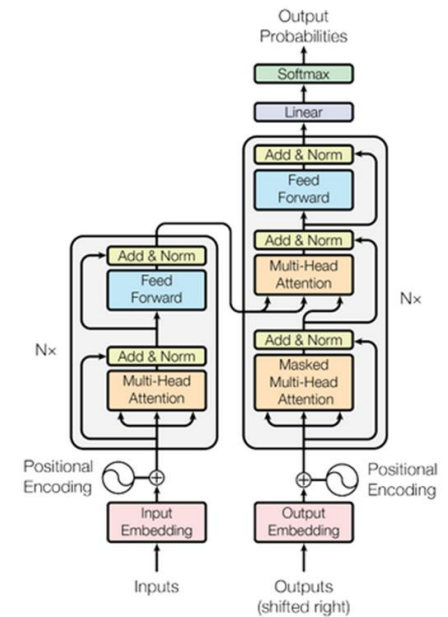
So, now we understand:

- Word Embeddings for transfer learning
- Encoding – Decoding mechanisms
- Attention to deal with alignment and complex long-distance relations.

Led to
the
ideas
of

Next Lecture

- Self-Attention
- Transformer Architecture (parallel processing)



Questions ...



Exercise 1: Decoding Strategies & The Bottleneck Problem (Greedy vs. Beam Search)

The lecture illustrates the limitations of standard RNN sequence-to-sequence generation and the distinction between greedy decoding and beam search. Without attention, BLEU scores collapse as sentence length increases beyond 20–30 words.

- Explain why a naive sequence-to-sequence model using a single fixed-dimension vector c (the final hidden state of the encoder) creates an information "bottleneck". What specific performance failure occurs on long sentences (e.g., 50 words) compared to short sentences?
- Contrast **Greedy Search** and **Beam Search** during decoding:
 - How does greedy decoding choose tokens at each target position, and why can this lead to suboptimal global sentence translations (e.g., "Yoda-like" phrasing)?
 - How does beam search with a beam width of $k \in [5, 10]$ mitigate this issue?
- Relate beam search decoding back to the **Label Bias Problem** discussed with MEMMs in Lecture 4. Why does optimizing path probabilities across a candidate beam prevent the decoder from being locked into locally high-probability errors?

Exercise 2: Computing Bahdanau Attention Weights & Context Vectors

The lecture shows attention as an information retrieval lookup over keys, values, and queries. Slide 19 details the mathematical computation of attention weights $\alpha_{t,i}$ and the dynamic context vector context_t . Suppose a decoder at time step $t = 1$ has generated a query vector q_1 from its previous state s_0 . The source sentence has 3 encoder states with corresponding key vectors k_1, k_2, k_3 and value vectors v_1, v_2, v_3 .

- Raw alignment scores: $\text{score}(q_1, k_1) = 2.0$, $\text{score}(q_1, k_2) = 1.0$, $\text{score}(q_1, k_3) = 0.0$
- Value representations: $v_1 = [1.0, 0.0]^T$, $v_2 = [0.0, 2.0]^T$, $v_3 = [1.0, 1.0]^T$

Questions:

- Using the softmax function $\alpha_{t,i} = \frac{\exp(\text{score}(q_t, k_i))}{\sum_j \exp(\text{score}(q_t, k_j))}$, compute the attention weights $\alpha_{1,1}$, $\alpha_{1,2}$, and $\alpha_{1,3}$ (round to two decimal places; use $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1.0$).
- Compute the resulting dynamic context vector $\text{context}_1 = \sum_{i=1}^3 \alpha_{1,i} v_i$.
- How does this dynamically computed context vector differ from the fixed memory vector in a standard RNN encoder-decoder model? Explain how error backpropagation flows through this weighted sum into the projection matrices W_Q, W_K, W_V during training.

Exercise 3: Word Alignment & Attention Matrix Interpretation

The lecture display attention heatmaps for English-to-French translation (e.g., *"The agreement on the European Economic Area was signed in August 1992"* → *"L'accord sur la zone économique européenne a été signé en août 1992"*).

- In the attention heatmap, what do the light squares versus the dark/black squares represent in terms of the numerical values of $\alpha_{t,i}$?
- Looking at the phrase *"European Economic Area"* translating to *"la zone économique européenne"*, observe how the attention weights deviate from a simple diagonal line. How does this demonstrate that attention directly solves the classical Machine Translation **alignment problem** and handles syntactic word-order permutations between languages?
- Why does the ability of the decoder to inspect all encoder hidden states at every generation step pave the way toward replacing recurrent connections entirely (leading to the **Transformer** architecture)?

Discussion



Maastricht University

Teaching note: Use today's ideas (the bottleneck problem, soft attention as lookup, word alignment) as concrete cases for the questions ahead — there's no single correct answer to these.

On the Information Bottleneck: Can a Fixed Vector Compress Infinity?

In a traditional sequence-to-sequence model, an encoder ingests a sentence of arbitrary length—whether 5 words or 80 words—and compresses its entire semantic content into a single fixed-size hidden state vector c before the decoder begins emitting output. The lecture shows that without attention, model translation quality plummets dramatically once sentences exceed 20 to 30 words.

Is it mathematically feasible for any finite, continuous vector space to store an unbounded amount of syntactic and propositional information without catastrophic loss?

If compressing a whole sentence into a single point in vector space creates an inescapable bottleneck, did the addition of Bahdanau attention simply enlarge that channel, or did it fundamentally alter what the encoder's job actually is?

On Soft Attention as a Database Query: Continuous Relaxation vs. Discrete Lookup

The lecture models the attention mechanism using the analogy of a fuzzy database query: a target query q is matched against candidate keys k_i , and the resulting softmax distribution weights the retrieval of values v_i . In traditional computer systems, looking up an address is discrete, deterministic, and exact.

- Why must neural attention be 'soft' and continuous rather than hard and discrete?
- What would happen to backpropagation and the gradient updates across W_Q , W_K , and W_V if the model simply picked the single highest-scoring key via an argmax instead of a softmax sum?
- Does computing a weighted blend of every single token in the sequence reflect genuine cognitive focus, or is it merely a mathematical compromise required to keep the training loop differentiable?

On the True Nature of Word Alignment: Grammatical Rules or Statistical Geometry?

In the attention heatmap for French-English translation, we observe the weights shift off the diagonal when translating 'European Economic Area' to 'la zone économique européenne', correctly swapping the adjective and noun positions.

Before neural attention, statistical machine translation (SMT) required explicit phrase-based distortion tables, and classical NLP used syntactic inversion rules to reorder phrases.

If the attention mechanism was given no explicit rules about French syntax or noun-adjective inversion, has the model discovered the formal grammatical structures Chomsky described, or has it merely learned a smooth geometric mapping between two sequential vector spaces?

If attention solves alignment purely through matrix multiplication and backpropagation, does modern deep learning prove that explicit grammatical syntax was an unnecessary abstraction all along?

